

LLM-Orchestrated Kinematic Adaptation (LOKA): An Immune System for Dynamic Humanoid Locomotion

Vineet Kulkarni

*School of Interactive Computing
Georgia Institute of Technology
Atlanta, Georgia, USA
vkulkarni46@gatech.edu*

Nitish Sontakke

*School of Interactive Computing
Georgia Institute of Technology
Atlanta, Georgia, USA
nitishsontakke@gatech.edu*

Dr. Sehoon Ha (Advisor)

*School of Interactive Computing
Georgia Institute of Technology
Atlanta, Georgia, USA
sehoonha@gatech.edu*

Abstract—Translating the semantic reasoning capabilities of foundation models into the high-frequency control signals required for robotic systems remains a critical challenge. Existing paradigms face significant architectural barriers: monolithic Vision-Language-Action (VLA) models struggle to adapt to novel task combinations in long-horizon domains; offline Domain Randomization (DR) leaves a persistent deployment gap under unexpected physical alterations; and autoregressive Large Language Model (LLM) code-generation frameworks introduce latencies that are fatal to dynamic bipedal balancing. This paper introduces LLM-Orchestrated Kinematic Adaptation (LOKA), a hybrid neuro-symbolic control architecture for real-time humanoid stabilization and gait discovery. LOKA decouples low-frequency semantic reasoning from high-frequency physical optimization through an asynchronous dual-rate execution engine. An intermediate Telemetry Compressor monitors motor effort and mission criteria, filtering raw high-frequency data into dense semantic trend tags. Upon detecting internal or environmental anomalies or an operator language request, a language model acts as an online system-identification engine via a structured YAML scratchpad. Rather than generating high-latency policy code, the orchestrator mutates the objective landscape, task targets, success criteria, and kinematic assumptions referenced by an active Model Predictive Controller (MPC). By bridging qualitative inference with quantitative optimal control, LOKA targets zero-shot physical adaptation under structural degradation and extreme environmental transitions.

Index Terms—Vision-Language Models, Embodied AI, Kinematic Adaptation, Model Predictive Control, Humanoid Robotics

I. INTRODUCTION

The pursuit of artificial general intelligence within the physical realm has catalyzed a transition from narrow control loops to broad, multi-modal generalist policies. At the forefront of this evolution is the integration of foundation models into robotic control architectures. However, recent evaluations reveal that current VLMs’ physical reasoning skills remain prone to causal hallucination and tracking drift [1]. Safely translating high-level cognitive reasoning into the continuous, high-frequency control signals required for dynamically balancing underactuated humanoids remains a formidable engineering challenge.

Current paradigms attempting to bridge this gap exhibit severe shortcomings. Monolithic Vision-Language-Action (VLA) models map perception directly to actions, yet they struggle to generalize behavior compositionally and fail when confronted with novel combinations of familiar skills, making them highly prone to compounding errors during long-horizon execution [3], [4]. Conversely, traditional trajectory optimization and Model Predictive Control (MPC) yield precise execution and constraint satisfaction over short horizons, but they are inherently rigid and lack global semantic awareness [5]. When an out-of-distribution physical shift occurs during live deployment, standard controllers fail to re-plan appropriately because their internal models are mathematically blind to the structural modification.

To handle these variations, state-of-the-art systems rely heavily on Domain Randomization (DR) [2]. DR is an offline robustification technique. While it accommodates bounded Gaussian noise in simulation, it cannot dynamically isolate and adapt to discrete, unmodeled physical variations during runtime. Recent online integration efforts utilize foundation models to synthesize reward functions via code generation [6]. Yet, even if code generation is moved outside the synchronous control loop to mitigate latency, the code-as-policy paradigm remains inherently brittle; ensuring syntactic and mathematical correctness without catastrophic failure remains a critical vulnerability [7].

To resolve the latency-stability paradox, this paper introduces LLM-Orchestrated Kinematic Adaptation (LOKA). LOKA moves the field away from high-latency code synthesis and toward real-time parameter and structural model mutation. The key contributions of this research are:

- **Effort-Driven Telemetry Compression:** An intermediate middleware node that continuously monitors actuator torque deficits, energy gradients, and mission error criteria. It delta-masks nominal behavior and synthesizes complex physical anomalies into compressed semantic strings, triggering the cognitive model before tracking errors lead to collapse.
- **Asynchronous Dual-Rate Execution:** A hardware-

capable architecture that completely isolates the high-frequency physical execution loop from the low-frequency inference time of the language model using thread-safe background processing queues.

- **Dynamic Kinematic Malleability:** A structured YAML scratchpad interface that enables the language model to act as a real-time system identification engine, granting it direct authority to mutate cost weights, planner numerics, task goals, success criteria, and the active controller’s physical self-awareness on the fly.

II. RELATED WORK AND LITERATURE REVIEW

To position LOKA within the current robotics landscape, we analyze related architectures across four primary research domains.

A. Slow-Fast Orchestration and Online Fault Recovery

Decoupling symbolic cognitive reasoning from reactive physical stabilization has been explored across multiple domains. The AESOP framework utilizes a high-frequency anomaly classifier to trigger a slow generative LLM reasoner, relying on an MPC to preserve physical feasibility during the reasoning latency gap on embedded hardware [8]. The REFLECT framework processes multisensory log data into a hierarchical natural language summary to enable an LLM to explain manipulation failures and guide corrective behavior [9]. While structurally related, REFLECT operates as a post-mortem diagnostic tool for general tasks, whereas LOKA’s Telemetry Compressor monitors high-frequency proprioceptive stream inputs to execute online adjustments.

Online parameter modification has been explored in wheeled robotics and autonomous driving. For instance, the Hey Robot! framework uses an LLM to modulate objective cost weights based on human preferences for mobile navigation, though it lacks structural awareness of the physical model [29]. Similarly, RobotxR1 exposes MPC cost structures and safety margins as online-tunable parameters that a closed-loop reinforcement learning agent alters via language prompts [10]. However, RobotxR1 treats the underlying kinematic model as an unchangeable prior. LOKA expands this authority by allowing structural modifications to the solver’s body model. The integration of neuro-symbolic planners, such as Recover, combines explicit rule-based logic with language agents to isolate physical faults [11]. LOKA avoids the execution gaps of pure symbolic rule bases by mapping outputs directly into a continuous optimal control loop.

Decoupling processing frequencies has also been applied to vehicle routing and monolithic transformers. AsyncDriver implements an asynchronous multi-threaded system to separate high-latency driving reasoning from real-time path planning [12]. In end-to-end foundation policies, Fast-in-Slow (FiS-VLA) builds a unified dual-system policy that embeds high-frequency execution token heads directly inside a larger, low-frequency VLM backbone [13]. LOKA shares these multi-rate design patterns but rejects black-box end-to-end actions,

ensuring low-level physical safety by anchoring the fast loop to a mathematically transparent MPC solver.

B. Language-to-Rewards and Code Generation

The dominant paradigm for bridging foundation models with optimal control relies on generating executable Python code. The Eureka and Text2Reward architectures utilize LLMs as automated programmers to search over and debug dense reward functions for reinforcement learning agents [14], [15]. While highly effective, these frameworks operate strictly as heavy offline synthesis engines requiring thousands of environment rollouts.

For online application, the ASPIRE framework distills successful debugging sessions into a reusable text-based skill library using synchronous programmatic synthesis for manipulation tasks [30]. Similarly, the Language to Rewards (L2R) framework translates natural language text descriptors into functional reward scripts optimized in a loop by a MuJoCo MPC [6]. Video2Reward derives reward metrics from target video behaviors for legged robots [16]. While expressive, these systems are fundamentally bounded by the autoregressive code generation bottleneck, exposing the system to syntax failures and unrecoverable latency during runtime anomalies. LOKA diverges sharply from this line of work by replacing code generation entirely with a strictly typed parameter and model mutation engine.

C. LLM-Guided Feedback Control Loops

A parallel vector of research integrates language models into active feedback control architectures. The LLMs-guided Adaptive Compensator utilizes an LLM to dynamically output an error-tracking compensator, verifying the loop via stability criteria on soft and humanoid links [17]. At the joint gain level, the LLM-Controller framework dynamically modulates nonlinear feedback properties while relying on the underlying baseline tracking architecture to preserve local stability [18]. Automated frameworks like ControlAgent and AgenticControl leverage multi-agent language systems to design classical proportional-integral-derivative (PID) layouts by reasoning through phase margins and settling times [19], [20].

To bridge optimization with neural safety, architectures such as Never Too Rigid to Reach enforce Lyapunov-constrained reinforcement learning priors to guide virtual model control on flexible arms [21]. Furthermore, the Meta-Control architecture synthesizes entirely new control topologies depending on changing operational skills [22]. LOKA shares the philosophy of leveraging language priors to modulate control characteristics, but shifts the intervention point. Rather than operating at the joint-level error feedback or gain layer, LOKA mutates the predictive horizon, task-space priorities, and kinematic assumptions of an online preview optimization engine.

D. Alternative Robot-Language Interface Modalities

Exposing intermediate control spaces to language models remains a highly active area of study. The SayTap framework

Fast loop: MuJoCo plant $\leftrightarrow$ MJPC

Telemetry buffer $\rightarrow$ Compressor / Error_Tracking

Anomaly or operator request $\rightarrow$ LLM orchestrator
(slow)

YAML scratchpad $\rightarrow$ costs / planner / tasks / belief

Affordance clamp $\rightarrow$ apply to live MPC

Fig. 1. LOKA dual-rate architecture (placeholder figure). A high-frequency MJPC loop continues while a background LLM mutates costs, planner numerics, task goals, mission criteria, and internal-model belief from compressed telemetry.

utilizes binary foot-contact matrices as a structured intermediate interface to map language commands to quadrupedal locomotion policies [23]. OmniVIC couples VLMs with retrieval-augmented generation to continually reconfigure the stiffness and damping properties of a Variable Impedance Controller (VIC), applying explicit force bounds for safety [24]. For manipulation, the IROSA framework maps user prompts to the operational parameters of Kernelized Movement Primitives (KMPs) through tool calls, completely isolating the language agent from direct trajectory generation [25].

To handle complex contact profiles, the CompliantVLA-adaptor and GenCHiP frameworks inject language-derived force compliance, variable impedance values, and compliance boundaries directly into manipulation matrices to ensure safety during physical interaction [26], [27]. LOKA generalizes this mapping philosophy to the full-body locomotion of under-actuated bipeds, utilizing a structured YAML configuration block to map semantic objectives directly into high-degree-of-freedom tracking priorities.

III. METHODOLOGY: THE LOKA ARCHITECTURE

LOKA operates as a continuous cognitive system-identification loop. The pipeline consists of four tightly coupled phases executed across a specialized multi-threaded computational architecture (Fig. 1).

A. Phase 1: Effort-Driven Telemetry Compression

Rather than triggering adaptation only after tracking error has corrupted the bipedal state vector, LOKA monitors energy expenditure within the low-level planner and mission criteria exposed by `Error_Tracking`. The MPC continuously generates an optimal control input vector representing commanded actuator torques. The Telemetry Compressor calculates the Effort Deviation Integral (E_{dev}) over a sliding temporal window of length W :

$$E_{dev} = \int_{t-W}^t \|u_{\text{actual}}(\tau) - u_{\text{nominal}}(\tau)\|^2 d\tau \quad (1)$$

When E_{dev} violates a safety boundary—or when composite mission excess exceeds a trigger threshold—the Compressor freezes the high-frequency telemetry buffer, delta-masks nominal state variables, and synthesizes the anomaly into

semantic tags. For example, a joint exhibiting maximum torque saturation but near-zero joint velocity is tagged as `[SATURATED_SEIZED]`, while high torque coupled with erratic velocity is tagged as `[SATURATED_FLAILING]`.

In the current implementation, compressed reports are organized into optional sections: (0) tracked state relative to live `Error_Tracking` terms; (1) cost / residual summaries; (2) motor-load asymmetry and saturation cues; (3) directive / mission context. We are additionally evaluating visual embeddings for spatial context. Sections can be toggled for ablations that isolate which observations the orchestrator requires.

B. Phase 2: Asynchronous Dual-Rate Execution

To prevent LLM inference latency from freezing the controller, LOKA implements a clean asynchronous partitioning scheme:

- 1) **Fast control loop (main thread):** Samples the true state vector from the physics engine (with future expansions accommodating state estimators), invokes the MPC planner step, maps actions to actuators, and updates rolling history queues.
- 2) **Slow cognitive loop (background thread):** When an anomaly is flagged, during a periodic heartbeat, or when an operator issues a natural-language request, a background worker constructs the semantic prompt—including prior interventions in the current failure episode—and dispatches it to the orchestrator.

While the LLM executes its forward pass, the main thread continues physics steps unhindered, relying on the last known valid configuration to keep the robot stable.

C. Phase 3: The YAML Scratchpad

The interface between the slow and fast threads is a strictly typed shared repository: the YAML scratchpad. The underlying MPC continuously minimizes a quadratic task-space cost over a preview horizon N :

$$J = \sum_{k=0}^{N-1} (\|x_k - x_{\text{ref}}\|_Q^2 + \|u_k\|_R^2) + \|x_N - x_{\text{ref}}\|_P^2 \quad (2)$$

where Q and R represent diagonal tracking and effort penalization matrices. The LLM mutates this optimization landscape by returning a formatted block with optional sections (omit keys that are unchanged):

- `Semantic_State`: chain-of-thought hypothesis and analysis before parameter edits;
- `Controller_Targets`: diagonal cost weights adjusting the importance of individual residuals;
- `Planner_Targets`: e.g., `agent_horizon` and `sampling_exploration` (applied by recreating the MJPC agent);
- `Task_Targets`: mission setpoints exposed by the active MJPC task (e.g., height and speed goals);
- `Error_Tracking`: composite trigger threshold plus weighted terms over `qpos/qvel` indices with

modes such as `below_target`, `above_target`, `abs_above`, and `abs_deviation`;

- **Model_Mutations:** continuous structural edits to the planner’s internal prior—actuator gear, geom friction, and body mass—resolved against MJCF names, independent of the hidden physical plant.

Multi-turn failure episodes surface prior diagnoses so the model must revise its hypothesis rather than repeat unchanged mutations. Operator requests may arrive outside failure and can rewrite task goals; when the mission changes, `Error_Tracking` must be updated to match.

D. Phase 4: Affordance Clamping and Execution

Upon completion of inference, the resulting string is captured via a thread-safe queue, parsed, and handed to the main loop. Before targets are applied, they pass through a deterministic affordance filter. If the language model proposes a parameter that violates hard kinematic constraints (e.g., a torso height beyond maximum leg extension), the filter clamps the value to the nearest physical boundary. Once validated, the main loop updates model properties and controller settings. If a joint is mutated to a gear ratio of 0.0 (virtual amputation), middleware zeros subsequent control signals for that actuator index, forcing the solver to discover alternative strategies with the remaining functional links. Mutations update what the MPC *believes*; they do not repair the physical plant. Policy guidance requires aligning belief with observations—e.g., reducing gear on a dead motor—rather than increasing gear to “force” a seized joint.

E. Algorithm Overview

The following pseudocode outlines the asynchronous orchestration loop.

```

1: Initialize:  $MPC_{state}$ ,  $LOKA_{memory}$ ,  $Q_{thread}$ 
2: Start Main Physics Loop (high frequency)
3: while Robot is Active do
4:    $E_{dev} \leftarrow \text{Calculate\_Effort\_Deviation}()$ 
5:    $e_{track} \leftarrow \text{Evaluate\_Error\_Tracking}()$ 
6:   if ( $E_{dev} > \text{Threshold}$  or  $e_{track} > \text{Trigger}$  or Operator-Request) and LLM not busy then
7:      $Telemetry \leftarrow \text{Compress\_State}(MPC_{state})$ 
8:      $\text{Spawn\_Thread}(\text{Query\_LLM}(Telemetry, \text{EpisodeHistory}), Q_{thread})$ 
9:   end if
10:  if  $Q_{thread}$  has message then
11:     $YAML_{updates} \leftarrow \text{Parse\_Queue}(Q_{thread})$ 
12:     $\text{Validate\_Constraints}(YAML_{updates})$ 
13:     $\text{Mutate\_MPC\_Model}(YAML_{updates})$ 
14:     $\text{Record\_Intervention}(YAML_{updates})$ 
15:  end if
16:   $u_{optimal} \leftarrow MPC.\text{Step\_Planner}()$ 
17:   $\text{Execute\_Torques}(\text{Mask\_Dead\_Actuators}(u_{optimal}))$ 
18: end while

```

IV. EXPERIMENTAL DESIGN AND EVALUATION

We evaluate LOKA in the MuJoCo physics simulation environment on an underactuated bipedal walker driven by MuJoCo MPC (MJPC), with planned extensions to additional humanoid embodiments.

A. Evaluation Scenarios

To validate robust reformulation across physical and semantic shifts, the system is subjected to four validation tasks:

- **Mass Shift:** An unmodeled mass of x kg (TODO) is attached asymmetrically mid-stride, forcing the system to detect effort imbalance and re-stabilize its center-of-mass target.
- **Friction Shift:** The global surface friction coefficient is stepped down from a nominal value of 1.0 to a reduced value (TODO) to evaluate real-time environmental adaptation.
- **Hardware Failure:** A hard fault is injected into a critical actuator (e.g., right hip), setting physical torque capacity to zero to validate the virtual-amputation pipeline.
- **Semantic / Operator Objectives:** The robot receives overarching language instructions (e.g., crouch and walk, crawl, hold upright) and/or must navigate a specialized corridor featuring a low overhanging ceiling followed by elevated rubble.

B. Quantitative Performance Metrics

The framework is evaluated using the following metrics:

- **Task Success Rate (TSR):** Fraction of trials where the robot reaches the terminal criterion without falling.
- **Time to Stabilization (T_s):** Duration between anomaly onset and restoration of a steady-state gait (or satisfaction of updated mission criteria).
- **Effort Deviation Integral (E_{dev}):** Accumulated energy metric in Equation 1, quantifying adaptation efficiency (alternative integrals over tracking residuals are also under consideration).
- **Outer-loop turns:** Number of orchestrator interventions until success or timeout.
- **Semantic Accuracy Percentage (SAP):** An automated judge (independent frontier model) comparing parsed scratchpad mutations against hidden fault metadata.
- **Interface reliability:** Rates of invalid YAML and unresolved MJCF names.

C. Comparative Baselines

TODO: Flesh out precise baseline implementations adapted to the dynamic bipedal domain (e.g., vanilla fixed MJPC, Domain Randomization without online mutation, ablations without model mutations or multi-turn history, and a code-generation / Language-to-Rewards variant where latency permits).

V. DISCUSSION AND LIMITATIONS

a) *Interpretability.*: Hypotheses and analyses in the scratchpad provide a human-readable audit trail of recovery strategies, complementing numeric weight and model edits.

TABLE I
PLACEHOLDER RESULTS (FAULT RECOVERY %, OPERATOR SUCCESS %, MEDIAN OUTER-LOOP TURNS).

Method	Fault (%)	Op. (%)	Turns
Fixed MJPC	–	–	–
Ablation (no mut.)	–	–	–
LOKA (full)	–	–	–

b) Latency and rate.: LLM calls are orders of magnitude slower than MPC. LOKA is therefore a supervisory layer for sparse events, not a replacement for the inner controller; asynchronous dual-rate execution is essential for stability during inference.

c) Safety and validity.: Structured parsing, MJCF name resolution, and affordance clamping reduce free-form action risk, but incorrect mutations can still destabilize the plant. Harder verification, sandboxing, and human approval remain important for hardware deployment.

d) Limitations.: Results will depend on prompt quality and the chosen LLM; hallucinations of invalid parameter names can still occur. The mutation vocabulary is intentionally small (gear, friction, mass). Current evaluation focuses on a walker embodiment in simulation, with humanoid transfer and richer spatial sensing still in progress. Some related citations remain placeholders pending final bibliographic cleanup.

VI. CONCLUSION

LOKA couples an LLM semantic orchestrator with MuJoCo MPC to adapt costs, planner settings, task goals, mission criteria, and internal-model belief under faults and operator intent. By compressing telemetry, isolating inference on a slow thread, and requiring multi-turn hypothesis revision through a typed YAML interface, LOKA aims to make adaptive locomotion both interpretable and operator-addressable—without the latency and brittleness of online code generation. Future work includes completing the evaluation matrix, multi-embodiment tasks, stronger verification of proposed mutations, closed-loop hardware trials, and richer dynamics-parameter identification within the same orchestration loop.

APPENDIX

A. System Prompt Formulation

TODO: Insert the baseline system prompt instructing the language model on the structure of the YAML scratchpad, available MPC tuning parameters, error-tracking policy, and model-mutation constraints.

B. Orchestrator Output Examples

TODO: Insert example YAML scratchpad blocks demonstrating zero-shot recovery solutions generated during the evaluation tasks (e.g., virtual amputation parameter modifications).

REFERENCES

- [1] Anonymous et al., “Probing the physical dynamics of Vision-Language Models,” *arXiv preprint arXiv:2510.06251*, 2025.
- [2] J. Tobin et al., “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IROS*, 2017.
- [3] M. Kim et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [4] Anonymous et al., “Unleashing More Actions via Action Compositional Training for VLA Models,” *arXiv preprint arXiv:2607.00351*, 2026.
- [5] Anonymous et al., “Analysis of Model Predictive Control in Changing Kinematic Environments,” *Placeholder Citation*, 2025.
- [6] W. Yu et al., “Language to Rewards for Robotic Skill Synthesis,” in *CoRL*, 2023.
- [7] Anonymous et al., “Evaluating the Brittleness of Code-as-Policy Frameworks in High-Frequency Domains,” *Placeholder Citation*, 2025.
- [8] Anonymous et al., “Real-Time Anomaly Detection and Reactive Planning with LLMs (AESOP),” in *RSS*, 2024.
- [9] Anonymous et al., “REFLECT: Summarizing Robot Experiences for Failure Explanation and Correction,” in *CoRL*, 2023.
- [10] Anonymous et al., “RobotxR1: Embodied Robotic Intelligence via Closed-Loop RL on LLMs,” *arXiv preprint arXiv:2505.03238*, 2025.
- [11] Anonymous et al., “Recover: A Neuro-Symbolic Framework for Failure Detection and Recovery,” *arXiv preprint arXiv:2404.00756*, 2024.
- [12] Anonymous et al., “AsyncDriver: Asynchronous LLM-Enhanced Planner for Autonomous Driving,” in *ECCV*, 2024.
- [13] Anonymous et al., “Fast-in-Slow (FiS-VLA): A Dual-System Foundation Model,” *arXiv preprint arXiv:2506.01953*, 2025.
- [14] Y. J. Ma et al., “Eureka: Human-Level Reward Design via Coding LLMs,” in *ICLR*, 2024.
- [15] Anonymous et al., “Text2Reward: Automated Dense Reward Function Generation for RL,” in *ICLR*, 2024.
- [16] Anonymous et al., “Video2Reward: Generating Reward Functions from Videos for Legged Robots,” *arXiv preprint arXiv:2412.05515*, 2024.
- [17] Anonymous et al., “LLMs-guided Adaptive Compensator,” *arXiv preprint arXiv:2507.20509*, 2025.
- [18] Anonymous et al., “LLM-Controller: Dynamic Robot Control Adaptation Using LLMs,” *Robotics and Autonomous Systems*, 2025.
- [19] Anonymous et al., “ControlAgent: Automating Control System Design via LLM Agents + Domain Expertise,” in *NeurIPS*, 2024.
- [20] Anonymous et al., “AgenticControl: An Automated Control Design Framework Using LLMs,” *arXiv preprint arXiv:2506.19160*, 2025.
- [21] Anonymous et al., “Never Too Rigid to Reach: Adaptive Virtual Model Control with LLM- and Lyapunov-Based RL,” *arXiv preprint arXiv:2510.22892*, 2025.
- [22] Anonymous et al., “Meta-Control: Automatic Model-Based Control Synthesis for Heterogeneous Robot Skills,” *arXiv preprint arXiv:2405.11380*, 2024.
- [23] G. Ji et al., “SayTap: Language to Quadrupedal Locomotion,” in *CoRL*, 2023.
- [24] Anonymous et al., “OmniVIC: Self-Improving Variable Impedance Controller with VLM In-Context Learning,” *arXiv preprint arXiv:2510.17150*, 2025.
- [25] Anonymous et al., “IROS: Interactive Robot Skill Adaptation using Natural Language,” *arXiv preprint arXiv:2603.03897*, 2026.
- [26] Anonymous et al., “CompliantVLA-adaptor: VLM-Guided Variable Impedance Action,” *arXiv preprint arXiv:2601.15541*, 2026.
- [27] Anonymous et al., “GenChiP: Generating Robot Policy Code for High-Precision Contact-Rich Manipulation,” *IROS*, 2024.
- [28] Anonymous et al., “LanguageMPC: LLMs as Decision Makers for Autonomous Driving,” *arXiv preprint arXiv:2310.03026*, 2023.
- [29] Anonymous et al., “Hey Robot! Personalizing Robot Navigation through MPC with an LLM,” in *ICRA*, 2025.
- [30] Anonymous et al., “Agentic Skill Programming through Iterative Robot Exploration (ASPIRE),” *NVIDIA GEAR Lab*, 2026.
- [31] Y. J. Ma et al., “DrEureka: Language Model Guided Sim-To-Real Transfer,” in *RSS*, 2024.